```
For every order-item record, display:

ProductID
SellingPrice
Average selling price of that product



select
  ProductID,
  SellingPrice,
  avg(SellingPrice) over(partition by ProductID) as avg_prd_sp
from `e1.order_items`



Display every order-item record and assign a row number based on
Total, from highest to lowest.

select
  OrderID,
  ProductID,
  Total,
  row_number() over(order by total desc) as sales_rank
from `e1.order_items`



For every product, assign a row number to its order-item records
based on sales from highest to lowest.
```

```
select
  ProductID,
  OrderID,
  Total,
  row_number() over(partition by ProductID order by Total desc)
as prd_row_number
from `e1.order_items`



| Product | Sales |
| ------- | ----: |
| P001    | 10000 |
| P002    |  9000 |
| P003    |  9000 |
| P004    |  7000 |



row_number() -->
p1->1
p2->2
p3->3
p4->4


rank()
p1 -> 1
p2 -> 2
```

```
p3 -> 2
p4 -> 4



dense_rank()
p1 -> 1
p2 -> 2
p3 -> 2
p4 -> 3




CTE --> Common Table Expression

with product_sales as (
  select
    ProductID,
    sum(Total) as total_sales
  from `e1.order_items`
  group by ProductID
)

select
  ProductID,
  total_sales,
  rank() over(order by total_sales desc) as sales_rank
from product_sales
```

```sql
with product_sales as (
  select
    ProductID,
    sum(Total) as total_sales
  from `e1.order_items`
  group by ProductID
)

select
  ProductID,
  total_sales,
  dense_rank() over(order by total_sales desc) as sales_rank
from product_sales
```

For every customer, show the number of orders they have placed, including customers with zero orders.

```sql
select
  c.CustomerID,
  c.Name,
  count(o.OrderID) as order_count
from `e1.customers` as c
left join `e1.orders` as o
on c.CustomerID = o.CustomerID
```

```
group by
  c.CustomerID,
  c.Name
order by order_count desc



Show every order with:

Customer name --> customers
Product name --> products
Category --> category
Quantity --> order_items
Selling price --> order_items
Total --> order_items



select
  c.Name,
  p.ProductName,
  ct.CategoryName,
  oi.Quantity,
  oi.SellingPrice,
  oi.Total
from `e1.customers` as c
inner join `e1.orders` as o
  on c.CustomerID = o.CustomerID
inner join `e1.order_items` as oi
  on o.OrderID = oi.OrderID
inner join `e1.products` as p
```

```
  on oi.ProductID = p.ProductID
inner join `e1.categories` as ct
  on p.CategoryID = ct.CategoryID



Customer name --> customers
Product name --> products
Category --> category
Quantity --> order_items
Selling price --> order_items
Total --> order_items



Sales by Category and Supplier

select
  cat.CategoryName,
  s.SupplierName,
  sum(oi.Total) as total_sales
from `e1.categories` as cat
inner join `e1.products` as p
  on cat.CategoryID = p.CategoryID
inner join `e1.suppliers` as s
  on s.SupplierID = p.SupplierID
inner join `e1.order_items` as oi
  on p.ProductID = oi.ProductID
group by
  cat.CategoryName,
```

```
  s.SupplierName
order by total_sales desc
```

How many different products has each customer purchased?

```
select
  c.CustomerID,
  c.Name,
  count(distinct oi.ProductID) as diff_prds
from `e1.customers` as c
inner join `e1.orders` as o
on c.CustomerID = o.CustomerID
inner join `e1.order_items` as oi
on o.OrderID = oi.OrderID
group by
  c.CustomerID,
  c.Name
order by diff_prds desc
```

The management wants to understand which product categories are generating the most revenue.

Using the products, categories, and order_items tables, calculate the total sales generated by each category.

```
Display:

CategoryID
CategoryName
Total Sales
Number of different products sold

Only include categories where the total sales are greater than
₹1,00,000.

Sort the result from highest to lowest total sales.



select
ct.CategoryID,
ct.CategoryName,
sum(oi.Total) as total_sales,
count(distinct p.ProductID) as no_of_prod
from `e1.categories` as ct
join `e1.products` as p
on ct.CategoryID=p.CategoryID
join `e1.order_items` as oi
on oi.ProductID=p.ProductID
group by
ct.CategoryID,
ct.CategoryName
having total_sales>100000
order by total_sales desc
```

```
The marketing team wants to identify customers who have
generated significant revenue.

Using the customers, orders, and order_items tables, calculate
the total amount spent by each customer.

Display:

CustomerID --> customers
Customer Name --> customers
City --> customers
Total Spent --> order_items

Only display customers whose total spending is greater than
₹50,000. --> having

Sort customers from highest spender to lowest spender. --> order
by


select
c.CustomerID,
c.Name,
c.City,
sum(oi.Total) as total_spent
from `e1.customers` as c
```

```
inner join `e1.orders` as o
on c.CustomerID = o.CustomerID
inner join `e1.order_items` as oi
on o.OrderID = oi.OrderID
group by
c.CustomerID,
c.Name,
c.City
having total_spent > 50000
order by total_spent desc


The procurement team wants to compare suppliers based on the
sales generated by the products they supply.

Using suppliers, products, and order_items, calculate:

SupplierID
SupplierName
Number of products supplied
Total sales generated by those products

Display only suppliers whose products have generated more than
₹2,00,000 in total sales.

Sort by total sales in descending order.



select
```

```
s.supplierID,
s.suppliername,
count(distinct p.ProductID) as products_supplied,
sum(oi.Total) as total_sales
from e2.suppliers as s
inner join e2.products as p
on s.supplierID = p.supplierID
inner join e2.order_items as oi
on p.ProductID = oi.ProductID
group by
s.suppliername,
s.supplierID
having total_sales > 200000
order by total_sales desc
```

The CRM team wants to understand the sequence in which customers placed their orders.

For every order, display:

CustomerID
OrderID
OrderDate
The order number for that customer

For example:

Customer C001
First order → 1
Second order → 2
Third order → 3

The numbering should be based on OrderDate, from oldest to newest.

The warehouse management team wants to classify products based on their current stock.

Using the inventory table, display:

WarehouseID
ProductID
Stock
Stock Status

Classify stock as:

Stock >= 200   → 'High Stock'
Stock 50 to 199   → 'Medium Stock'
Stock < 50   → 'Low Stock'

Sort the result by WarehouseID and Stock from highest to lowest.

